One-Cycle Repository Planning for Horror$Place VM-Constellations

Overview

This report describes a planning and file-structure strategy that aims to make each repository change a one-cycle iteration: a single, coherent pass that yields implementation-ready files without needing follow-up restructuring or major rewrites.
It builds on the VM-constellation architecture, the sovereign Horror.Place core, and the Tier-2 vault designs such as HorrorPlace-Atrocity-Seeds, emphasizing schema-first contracts, registries, and GitHub-native workflows.[1][2]
The goal is to turn the file layout and pipeline rules themselves into machine-enforceable constraints on AI and human contributors, so that any new work product is already aligned with invariants, metrics, and cross-repo wiring.

Doctrine: One-File, One-Cycle, Contract-First

The VM-constellation doctrine already encodes a "one-file-per-response" rule for AI agents in the public core, where each generation must declare its repository, path, schema, safety tier, and invariant usage.[2]
Extending this to a one-cycle repository iteration requires tightening the notion of contract-first development: every new or modified file must be fully described and validated against schemas, registries, and workflows before being merged.[1]
This shifts iteration from ad-hoc refactors to planned, atomic contract changes: schemas evolve rarely and in controlled branches; instances

and content follow those contracts but do not reshape structure opportunistically.

Under this doctrine, a "cycle" for a repository is defined as: plan contracts and structure; define or update schemas if absolutely necessary; generate or edit instances (events, regions, personas, styles); update registries; run end-to-end validation and orchestration workflows.
If any step fails, the cycle is aborted and retried from the planning stage, rather than patched locally in the middle; this enforces clean, auditable iterations similar to database migration discipline.

File-Structure Planning Principles

Mirror the Constellation Pattern

The VM-constellation blueprint and Atrocity-Seeds audit provide a canonical directory pattern: schemas/, contracts or instances/ (events, regions, personas, styles), registry/, constellation/ docs, and optional implementation code (src/, engine/, scripts/).[2][1]
The public Horror.Place repo mirrors this with schemas in schemas/, specs and blueprints in docs/, engine code under src/ and engine/, and registries under registry/.[3][2]
One-cycle planning starts by enforcing that every repository in the VM-constellation conforms to a subset of this pattern, with explicit roles:

- Tier 1 (Horror.Place): schema definitions, invariant and metric APIs, style contracts, persona/event/region contracts, registries, and orchestration specs.
- Tier 2 vaults (e.g., Atrocity-Seeds, Black-Archivum, Codebase-of-Death, Spectral-Foundry): instance contracts (seeds, bundles, personas), private registries, PCG/audio/AI implementation code, constellation docs, and CI workflows.
- Tier 3 labs (e.g., Redacted-Chronicles, Neural-Resonance-Lab, Dead-Ledger-Network): research schemas, telemetry data, cryptographic circuits, lab registries, and experimental protocols.[3][2]

Planning the file-structure for a new repo or a major iteration therefore begins with mapping it to one of these roles and instantiating the canonical skeleton.

Separate Shape, Behavior, and Discovery

Horror.Place and its blueprints emphasize a clear separation between shape (schemas), behavior (contracts specifying invariant and metric envelopes, implication rules, and phase structure), and discovery (registries mapping IDs to files, hashes, and tier descriptors).[2][3]
One-cycle iteration requires treating these as independent but linked axes: schema files evolve according to a versioned migration plan; contract/instance files are added or updated within those schemas; registries are the sole source of truth for discovery and cross-repo integration.
For example, `schemaseventcontractv1.json` defines the permissible fields for event contracts; `events/xxx.json` provide instances like `araldissolutionv1.json`; `registry/events.json` indexes those instances with IDs, paths, hashes, safety tiers, and optional Dead-Ledger references.[1]

Planning actions must ensure that any change to behavior or content is expressed as:

1. A schema update (only when necessary), versioned and documented.
2. One or more contract files conforming to that schema.
3. A registry update that makes these contracts visible to orchestration workflows and downstream engines.

Enforce Tier-Appropriate Content Boundaries

The sovereign blueprint and History Entertainment Framework make clear that Tier 1 repos must remain implication-only and GitHub-safe, while Tier 2 and Tier 3 hold sensitive trauma archives, agent implementations, and BCI or cryptographic data.[3][2]
Planning file structures with one-cycle intent therefore starts by stating explicit content boundaries in each repository's `README` and constellation docs: what can and cannot be stored here, which schemas apply, and how references to other tiers are encoded (e.g., invariant bundle IDs from Black-Archivum, IPFS hashes of generated

assets, ZKP circuit IDs in Dead-Ledger-Network).[1][2]
These boundaries become constraints on file generation and commit rules, ensuring that contributors never need to retroactively move files across tiers because the rules were unclear.

Pre-Committed Coding Steps per Repository

Horror.Place (Tier 1 Public Core)

For the public Horror.Place repo, a one-cycle iteration should follow a pre-committed sequence:

1. **Schema and API planning**: decide whether changes are required to core schemas (invariantsv1.json, entertainmentmetricsv1.json, stylecontractv1.json, eventcontractv1.json, personacontractv1.json) or Lua/Rust APIs like H. invariants and Telemetry metrics.[2][3]
2. **Spec updates**: update or add docs in docs/ (e.g., hphistoryentertainmentframework.md, spectrallibraryspec.md, style router specs) to describe the change and its rationale in terms of invariants and metrics.[3]
3. **Schema implementation**: modify or add schema files under schemas/ with versioned titles and explicit compatibility notes.
4. **Engine code alignment**: adjust src/ and engine/ modules (spectrallibrary.rs, horrorinvariants.lua, pcggenerator.rs, audioautomation.rs) to respect the updated schemas and APIs, but never embed content.[3]
5. **Registry wiring**: ensure registry/*.json are updated if new contract IDs, styles, events, regions, or personas are introduced at Tier 1.
6. **CI and linting**: run or update GitHub Actions workflows that validate style contracts, schemas, and registry integrity, plus static analysis for invariant and metric usage.[2]

The planning document for each cycle (e.g., docs/core-iteration-plan-YYYYMMDD.md) should list exactly which of these steps are in scope, which files are touched, and which invariants and metrics are being extended or constrained.

HorrorPlace-Black-Archivum (Tier 2 Vault)

For Black-Archivum, which holds archival records and invariant bundles, pre-committed steps per cycle focus on data normalization and bundle contracts.[4][2]
Each cycle should:

1. Add or extend archiverecordv1.json-conforming files in data/archives/, documenting sources and transformation steps.
2. Generate or extend invariant bundles in contracts/invariantbundles/ (e.g., invariant-bundle-aral-sea-v1.json), tying archival records to invariant vectors (CIC, AOS, SHCI, etc.).[4]
3. Update registry/invariants.json with new bundle IDs, paths, hashes, safety tiers, and Dead-Ledger references.
4. Update constellation docs explaining how these bundles will be consumed by Atrocity-Seeds, Spectral-Foundry, and Codebase-of-Death via the H. API.[1][2]
5. Run CI workflows that validate archival records, bundle schemas, hashes, and Dead-Ledger anchoring.

Because Black-Archivum is upstream for many other repos, schema changes here are rare and should be front-loaded into dedicated cycles with clear migration paths.

HorrorPlace-Atrocity-Seeds (Tier 2 Vault)

The Atrocity-Seeds Tier-2 vault has already been specified as a PCG seed repository with a refined structure: schemas/ (event, region, gore-assembly style schemas), events/, regions/, registry/, gore-assembly/, and constellation/.[1]
A one-cycle iteration for Atrocity-Seeds must:

1. Confirm or extend schemas: eventcontractv1.json, regioncontractv1.json, goreassemblystylev1.json, ensuring they mirror Tier 1 schemas where applicable.
2. Add or modify event contracts in events/ (e.g., araldissolutionv1.json) and region contracts in regions/ (e.g., basinofsilencev1.json, region-scoped ALN encounter patterns), always referencing invariant bundles and specifying metric intent.[1]

3. Extend gore-assembly/ assets as implication-only style and state descriptors consistent with goreassemblystylev1.json, never adding explicit content.[1]
4. Update registry/events.json and registry/regions.json with new IDs, paths, hashes, safety tiers, intensity bands, and any Dead-Ledger or public descriptor references.[1]
5. Update constellation/*.md docs (e.g., pipeline and constellation notes) if workflows or external interfaces change.
6. Run all CI workflows that validate contracts against schemas, compute hashes, and ensure registries remain consistent.

Planning for this repo should treat every new seed (event or region) as a micro-cycle: design invariant preconditions and metric envelopes; draft the contract; wire it into registries; validate against upstream bundles; then make it discoverable to Horror.Place orchestrators.

Other Tier-2 and Tier-3 Repositories

Repositories such as Codebase-of-Death, Spectral-Foundry, Liminal-Continuum, Redacted-Chronicles, Neural-Resonance-Lab, and Dead-Ledger-Network already have documented skeletons with schemas, contracts, docs, and registries.[2][3]
One-cycle planning for these follows the same pattern: decide which schemas to evolve, which contracts or artifacts to add, how registries must change, and which workflows and cryptographic checks will be exercised.
For Dead-Ledger-Network, cycles will focus on ZKP schemas, circuits, verifier registries, and protocol docs; for Redacted-Chronicles and Neural-Resonance-Lab, they will emphasize telemetry schemas, dataset manifests, and analysis pipeline specs.[2]

GitHub Workflows as Cycle Enforcers

Declarative Triggers and Reusable Jobs

The VM-constellation pipeline architecture specifies GitHub Actions workflows as the orchestrated backbone that connects repositories, with events like push, pull_request, workflow_call, repository_dispatch, and schedule used to validate and propagate

changes.[2]
A one-cycle iteration designates a small set of workflows as authoritative for each repository, such as validate-schemas.yml, update-registry.yml, and dispatch-to-vaults.yml in Horror.Place, or validate-event.yml and generate-region.yml in Atrocity-Seeds.[2][1]
Workflows are written to be idempotent and isolated: running them multiple times with the same inputs does not corrupt registries or duplicate entries, and each run happens in a clean environment.[2]

Planning actions include:

- Enumerating all workflows and aligning them with the schema/contract/registry triad.
- Ensuring each workflow begins with cryptographic verification where applicable (e.g., GPG signature checks, hash validation) before performing updates.
- Using reusable workflows and matrix strategies to avoid duplication and allow multi-repo sync operations.

From Contract Change to Cross-Repo Propagation

The sovereign blueprint and orchestrated automation spec describe a cross-repo pattern where a change to a Tier-1 registry triggers workflows in Tier-2 vaults, which then produce artifacts and send back IPFS hashes and status.[2]
One-cycle iteration extends this by treating registry changes as the canonical start and end of a cycle: no event or region contract is considered live until its registry entry is validated and, where required, downstream workflows have run to completion.
For example, when registry/events.json in Horror.Place gains a new event ID pointing at Atrocity-Seeds, the orchestrator workflow sends a repository_dispatch to Atrocity-Seeds, which validates and processes the corresponding seed, then returns confirmation and artifact hashes for inclusion in public registries.[1][2]

This pattern ensures that multi-repo changes are coordinated through declarative contracts rather than ad-hoc scripts, and that a single cycle can move cleanly from spec to implementation to registry update.

Research Actions to Improve One-Cycle Logic

Internal Research Across Existing Blueprints

Several existing documents already suggest internal research directions:

- Deepening the design of PCG modules that maintain atrocity anchors and history-weighted layouts based on CIC, HVF, and LSG, ensuring that map generation can be driven entirely by invariant contracts.[3]
- Refining style contract DSL usage in AI authoring templates so that all narrative, visual, and audio generation is constrained by explicit implication rules and invariant/metric bands.[3]
- Enhancing registry schemas to include more detailed provenance and telemetry summary fields, allowing cycles to be evaluated and prioritized by their entertainment metric impact (UEC, EMD, STCI, CDL, ARR).[3]

Planning each iteration should include a research step that reviews these blueprints, identifies gaps or ambiguities, and updates schemas and docs before any new implementations are added.

External Research on Contract-Driven Development

To strengthen the generality and future reuse of this one-cycle planning logic (including for non-horror VM-constellations), additional research actions should focus on:

- Studying contract-driven and schema-first development practices (e.g., JSON Schema/OpenAPI-based contract testing, consumer-driven contracts) to formalize evolution patterns for schemas and registries.
- Investigating infrastructure-as-code and GitOps patterns, where repository state and workflows are treated as declarative specs, and changes are promoted through environments via automated checks.
- Examining monorepo vs multi-repo architectures and best practices in large, federated systems to refine how the VM-

constellation coordinates changes without losing independence.

These research strands would help formalize a cross-domain pattern: any VM-constellation-like system can define invariants, metrics, schemas, and registries, then structure repositories and pipelines around one-cycle iterations.

Telemetry-Driven Iteration Decisions

The History Entertainment Framework ties invariant fields (CIC, MDI, AOS, RRM, FCF, SPR, RWF, DET, HVF, LSG, SHCI) to entertainment metrics (UEC, EMD, STCI, CDL, ARR) and expects telemetry from real players or testbeds to drive evolution.[3]
Future research actions should therefore explore how iteration planning can use aggregated telemetry to select which contracts, regions, styles, or personas to update in the next cycle.
For example, regions consistently showing low UEC and EMD or excessive CDL could be flagged in registries for targeted redesign, while preserving adherence to historical invariants; this would make cycles data-driven rather than intuition-driven.

Naming and Formalizing the Pattern

Given its emphasis on contracts, invariants, and single-pass repository evolution, this methodology could be named to make it portable:

- **Invariant-Cycle Planning (ICP)**: each cycle is planned around invariant schemas and bundles, ensuring history-grounded consistency.
- **Contract-Closed Iteration (CCI)**: iterations begin and end with contract and registry updates, closing the loop without ad-hoc fixes.
- **Constellation Single-Pass Protocol (CSP)**: a protocol for multi-repo VM-constellations to apply one-pass, cross-tier changes.

Additional research should evaluate which name best communicates the intent and maps cleanly onto similar patterns in other ecosystems (e.g., DDD bounded contexts, schema-registry-based streaming systems), and then codify it in a public document (e.g.,

docs/constellation-iteration-doctrine.md) for inclusion in Horror.Place.[2]

Reuse for Future VM-Constellation Work

The proposed planning and file-structure logic, once named and documented, can be treated as a template for any future VM-constellation-style project.
New constellations would define their own invariant sets, metrics, schemas, and tiers, then adopt the same pattern: canonical repo skeletons, schema/contract/registry separation, GitHub-native orchestrated workflows, and telemetry-informed iteration planning.[3][2]
Because the Horror$Place design already demonstrates how such a system can be tied to historical data, cryptographic proofs, and BCI-driven metrics, it offers a strong reference for generalizing one-cycle iteration to other genres and domains.

References

1. verifying-the-history-to-encou-SBAy_UkvSxKN_dcR1HCoZg.md - A refined, doctrine-aligned structure for HorrorPlace-Atrocity-Seeds is text HorrorPlace-Atrocity-Se...
2. a-new-addition-to-the-rivers-o-kBKpoeZeQ9mj8PyJlgtxSg.md - img srchttpsr2cdn.perplexity.aipplx-full-logo-primary-dark402x.png styleheight64pxmargin-right32px
3. this-research-focuses-on-advan-nfmYmGc6R5iLLtRfYigbWw.md - Styles are cryptographically sober contracts between the history layer, the engine, and AI tools, no...
4. a-declarative-framework-for-th-L5mG.NwsQNGelqIzZT8Seg.md